

<!--

- @Date: 2021-12-29 21:44:25

- @Author: bFeng

-->

二叉树展开为链表

Category Difficulty Likes Dislikes

algorithms Medium (72.69%) 1016 -

Tags

tree | depth-first-search

Companies

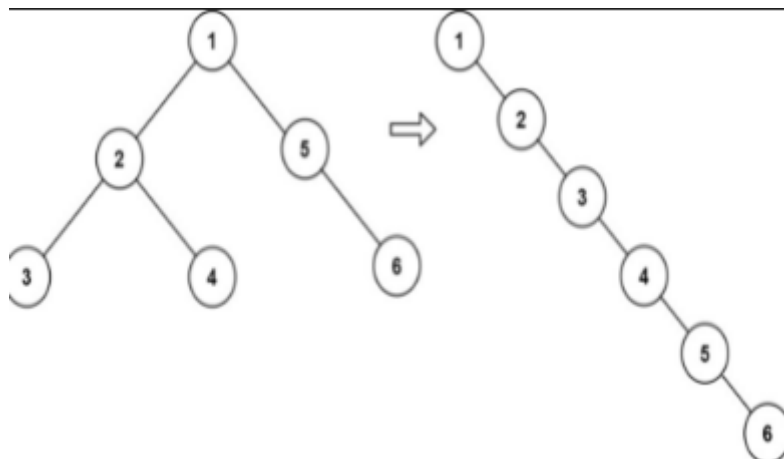
microsoft

给你二叉树的根结点 `root`，请你将它展开为一个单链表：

展开后的单链表应该同样使用 `TreeNode`，其中 `right` 子指针指向链表中下一个结点，而左子指针始终为 `null`。

展开后的单链表应该与二叉树 先序遍历 顺序相同。

示例 1：



输入：root = [1,2,5,3,4,null,6]

输出：[1,null,2,null,3,null,4,null,5,null,6]

示例 2：

输入：root = []

输出：[]

示例 3：

输入：root = [0]

输出：[0]

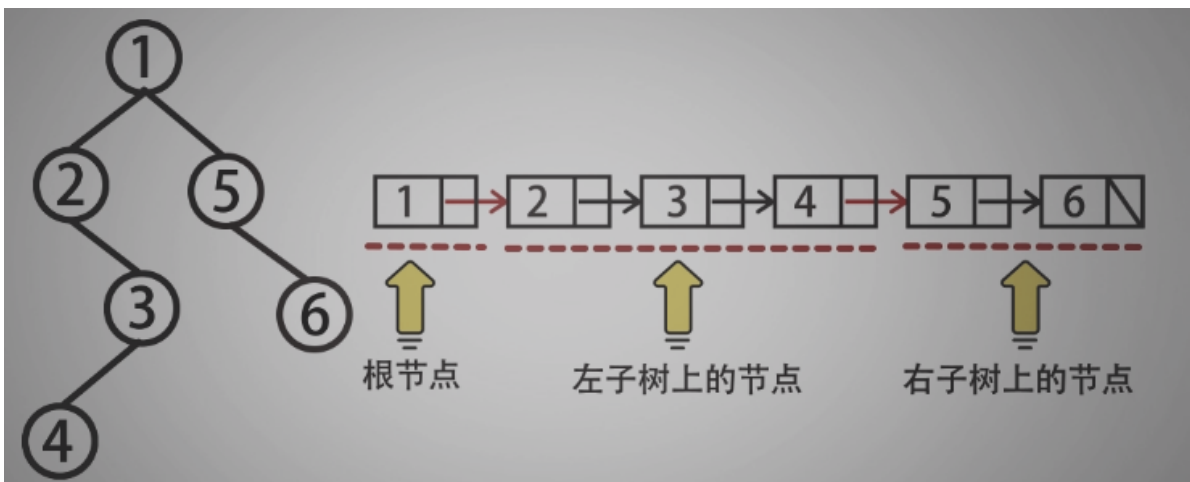
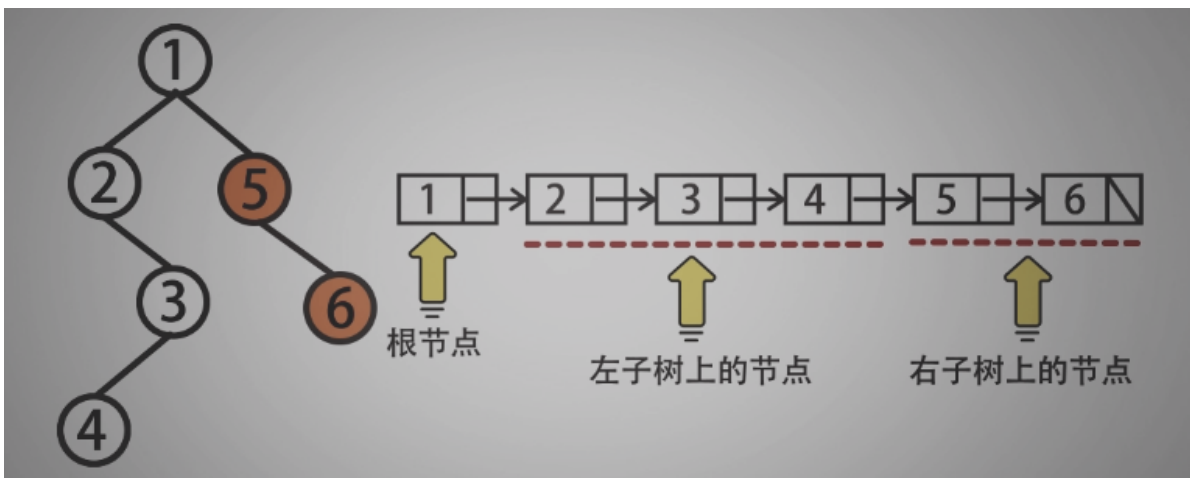
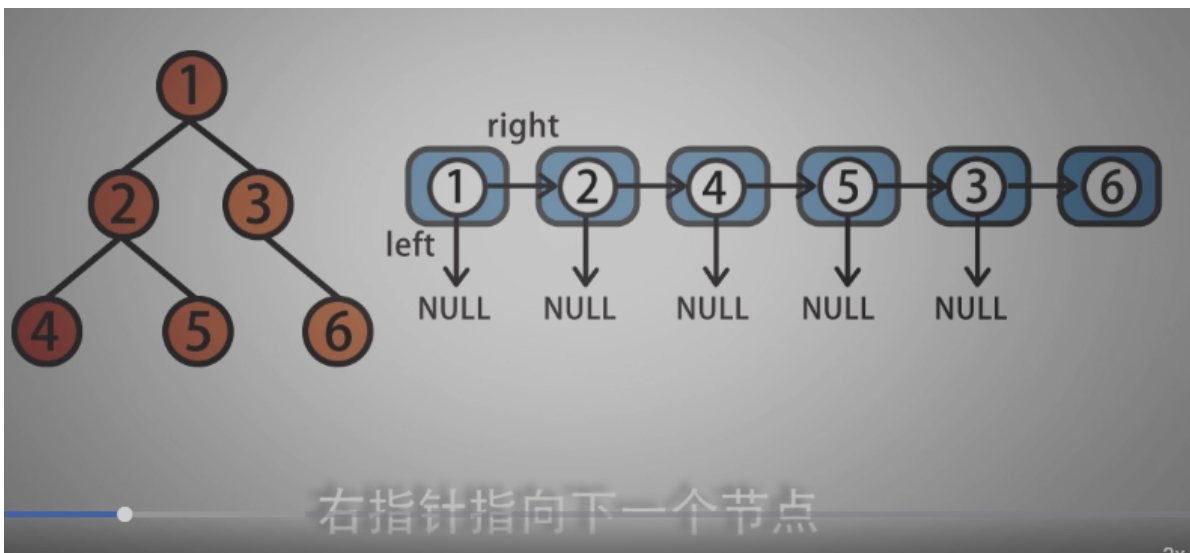
提示：

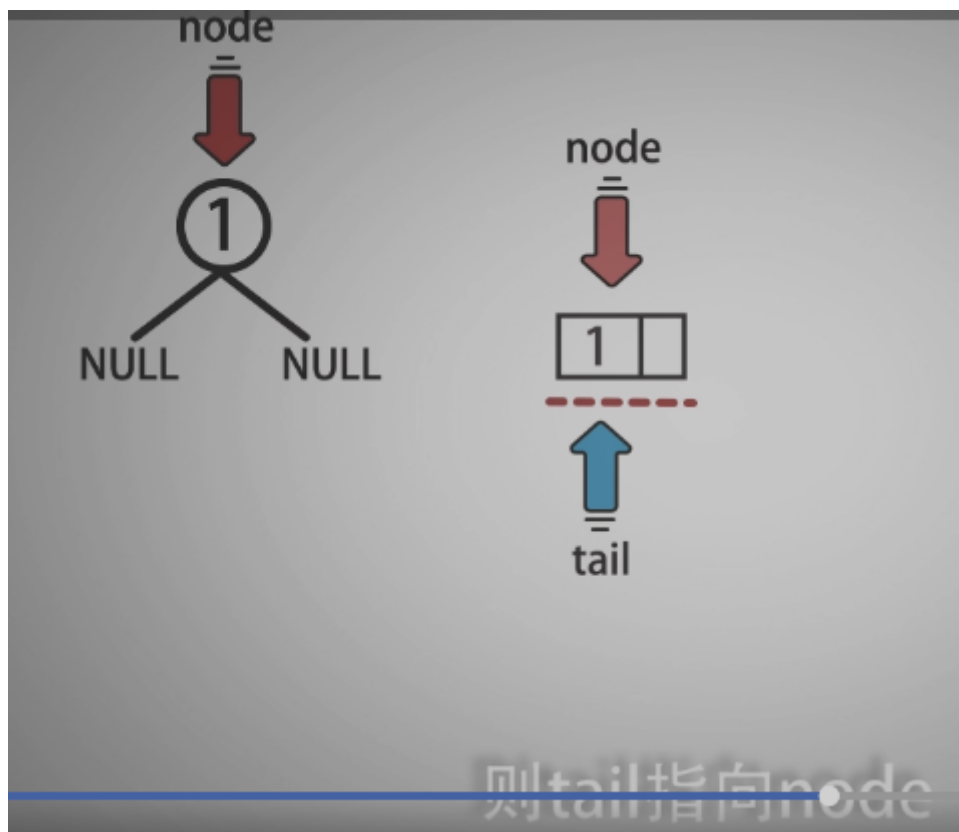
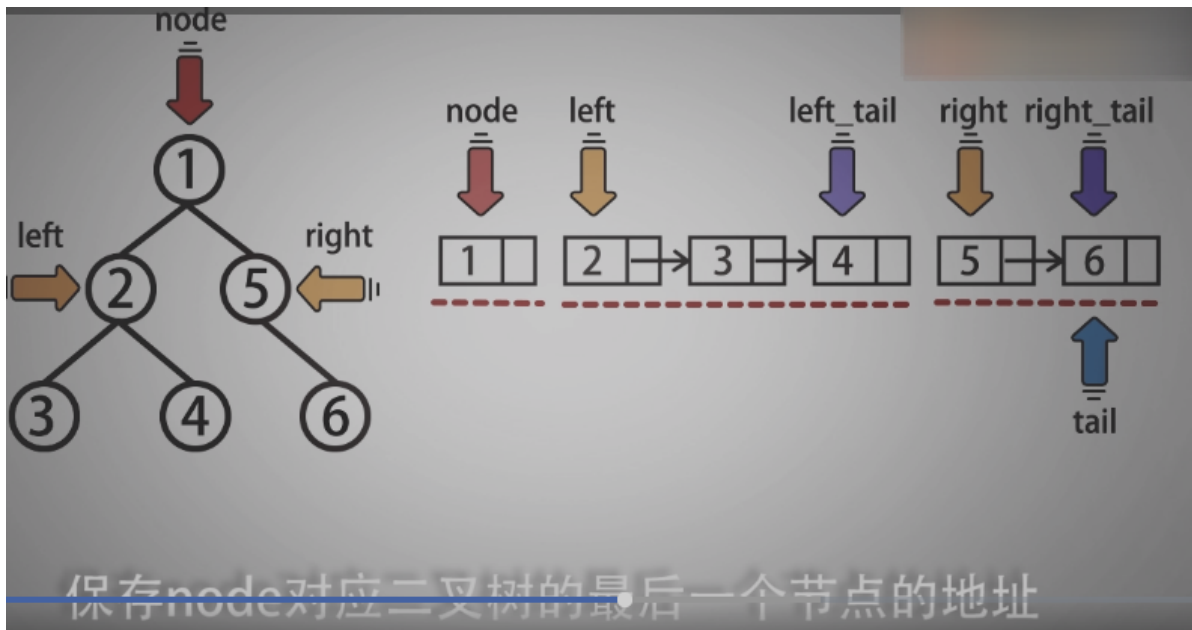
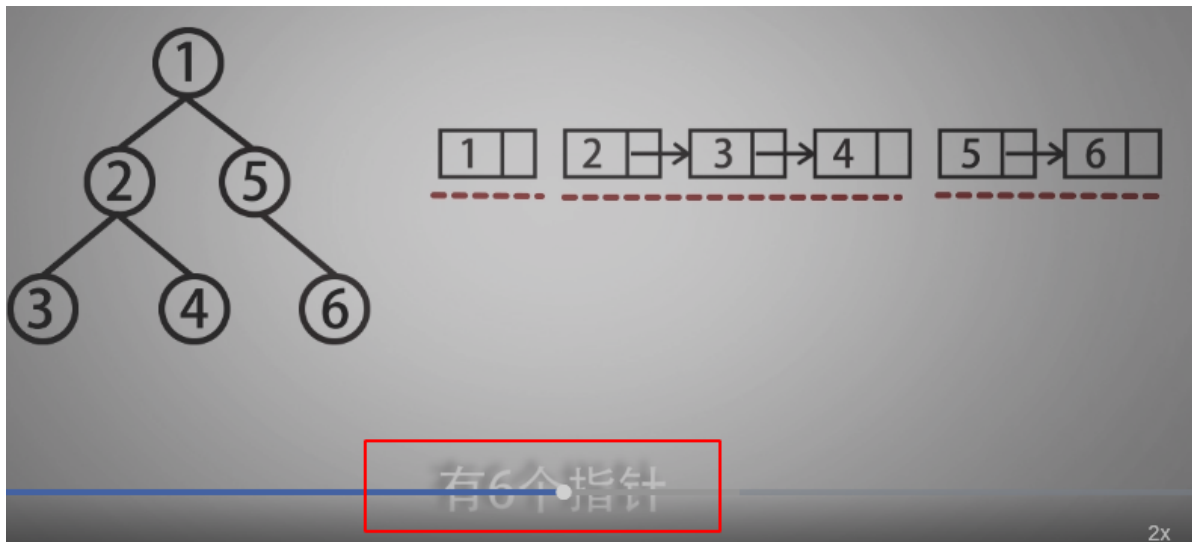
树中结点数在范围 [0, 2000] 内

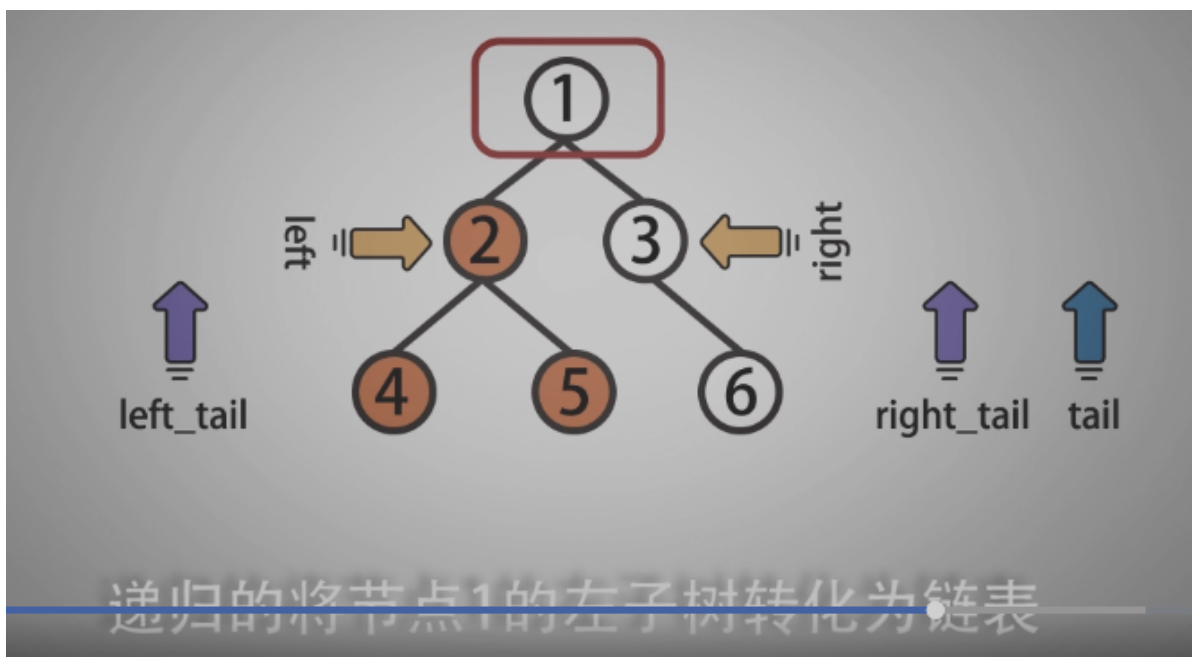
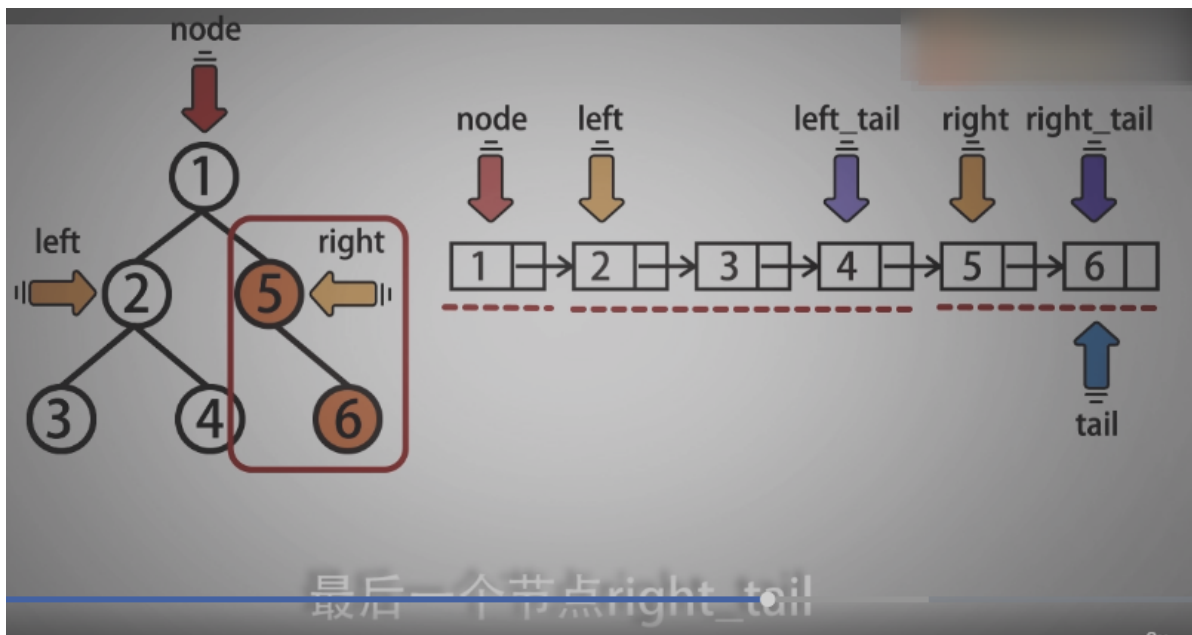
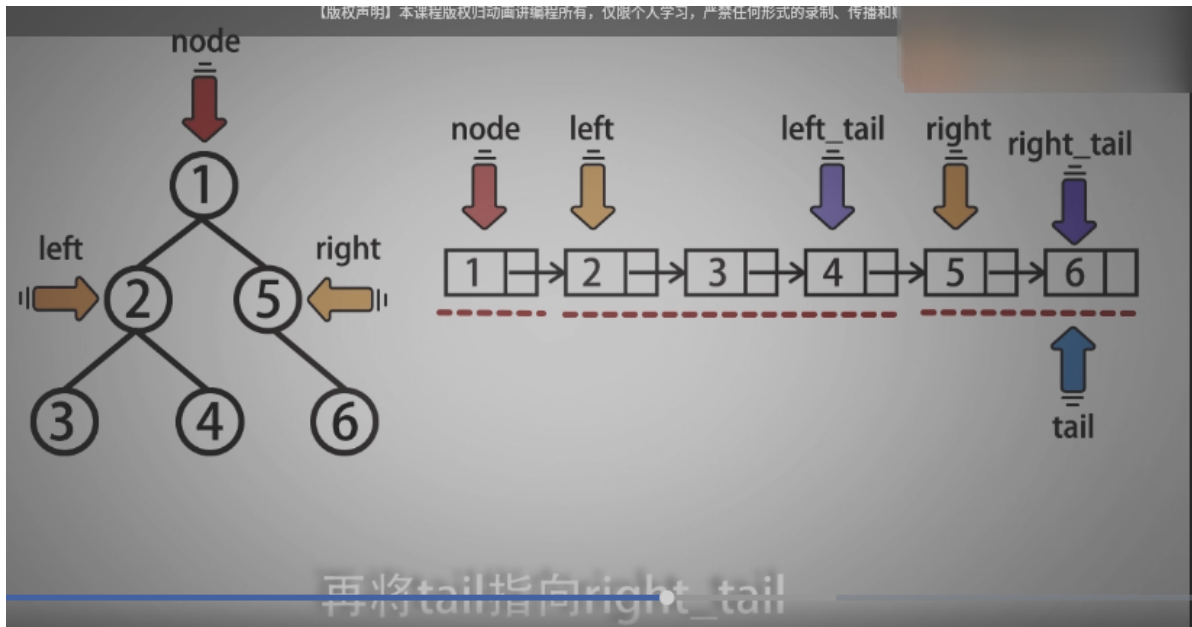
$-100 \leq \text{Node.val} \leq 100$

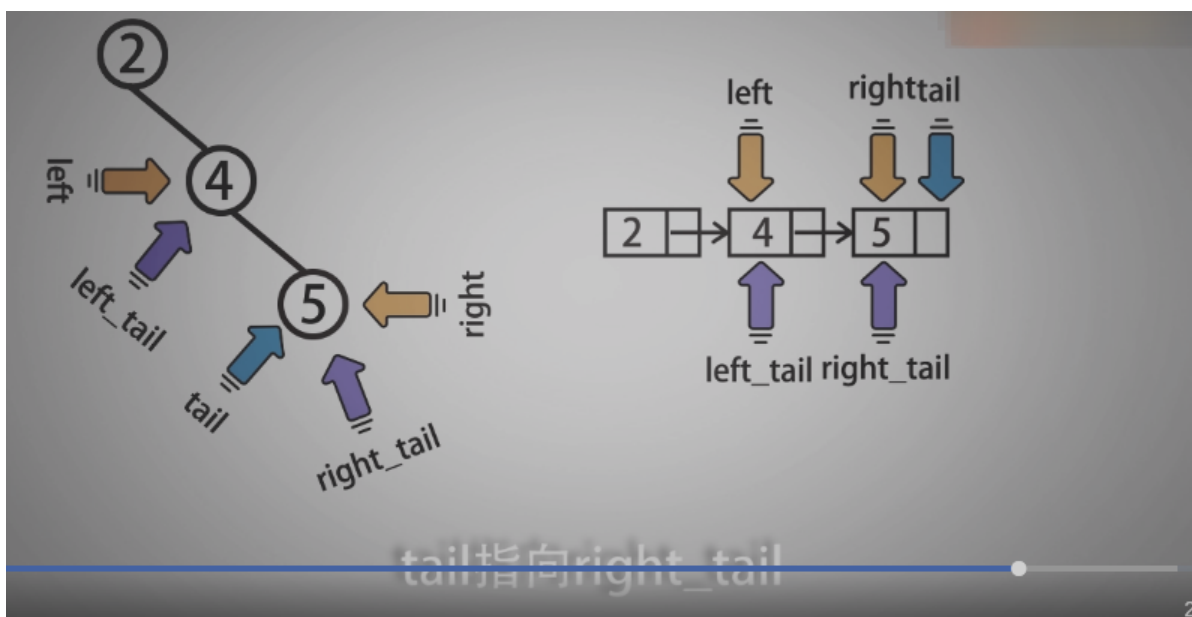
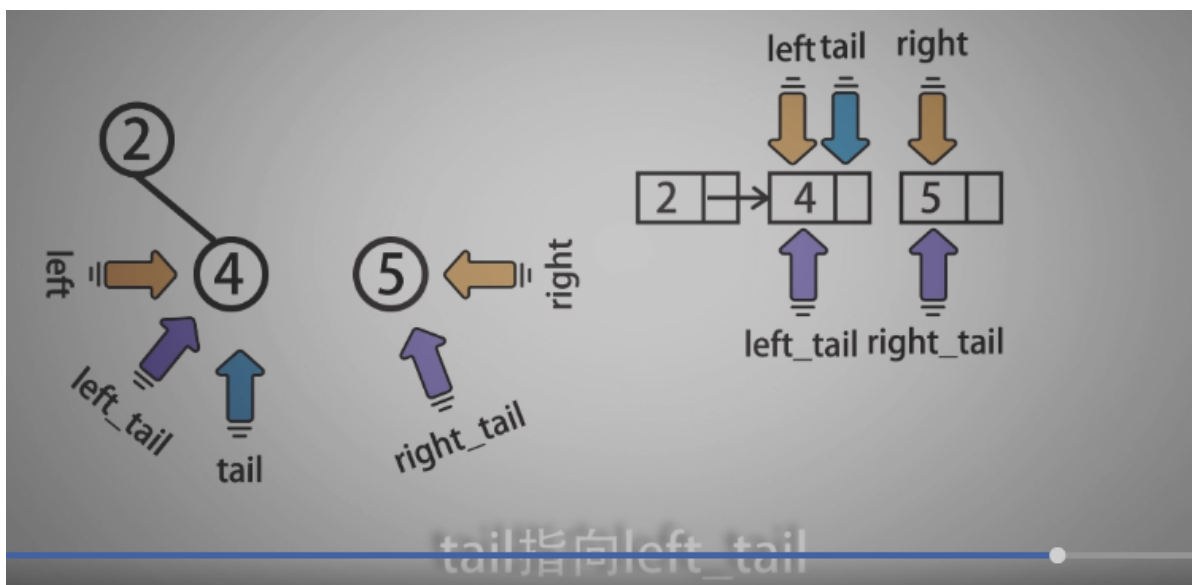
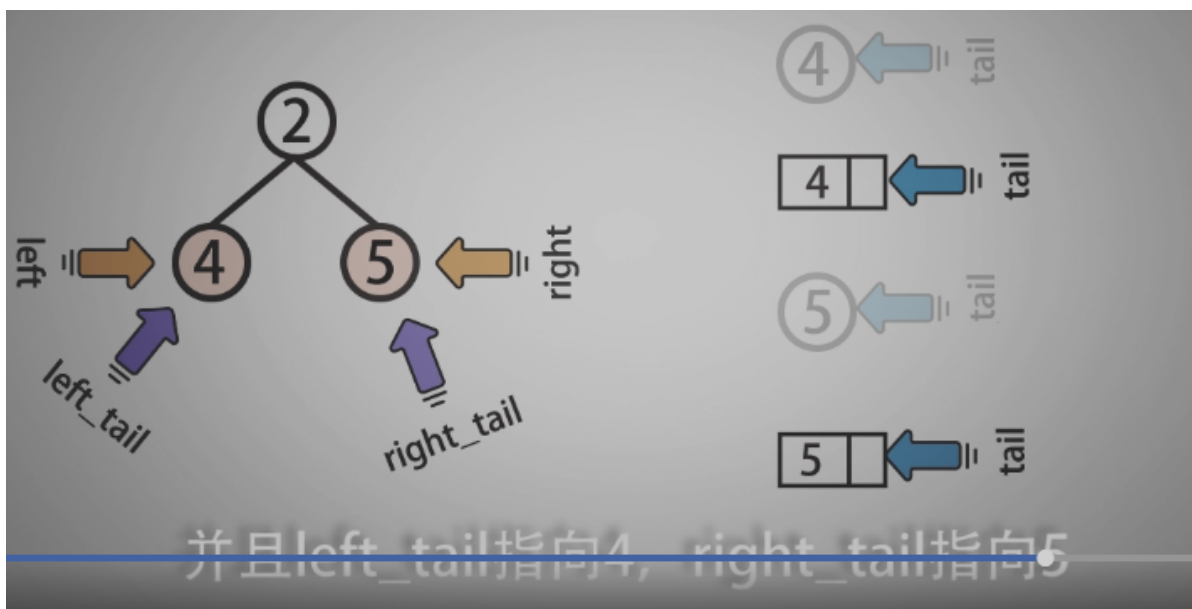
进阶：你可以使用原地算法（O(1) 额外空间）展开这棵树吗？

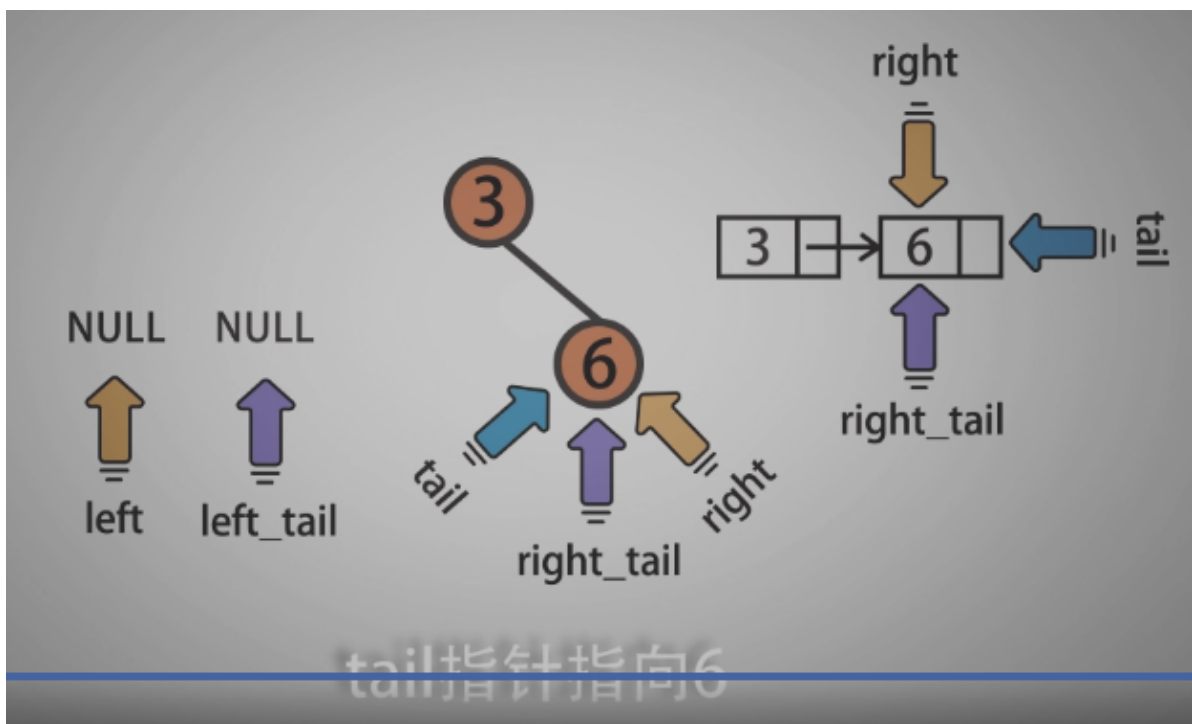
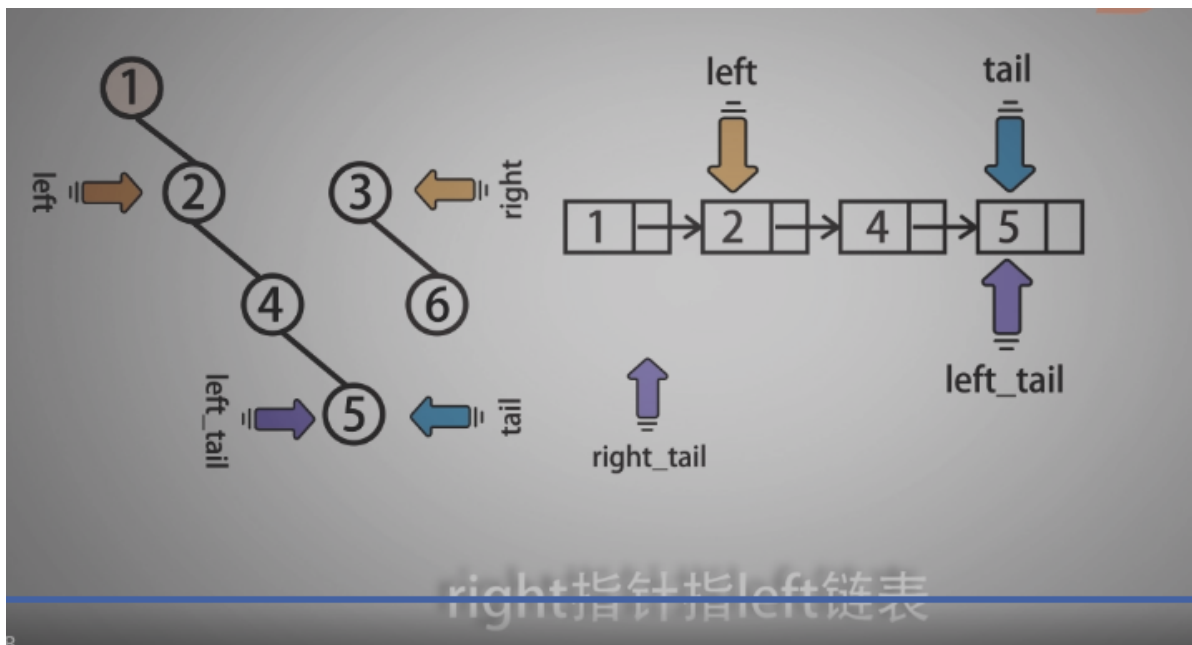
Discussion | Solution











```

1  /*
2   * @Date: 2021-12-29 21:46:30
3   * @Author: bFeng
4   */
5  /*
6   * @lc app=leetcode.cn id=114 lang=cpp
7   *
8   * [114] 二叉树展开为链表
9   */
10
11 // @lc code=start
12 /**
13  * Definition for a binary tree node.
14  * struct TreeNode {
15  *     int val;

```



```

16     *   TreeNode *left;
17     *   TreeNode *right;
18     *   TreeNode() : val(0), left(nullptr), right(nullptr) {}
19     *   TreeNode(int x) : val(x), left(nullptr), right(nullptr) {}
20     *   TreeNode(int x, TreeNode *left, TreeNode *right) : val(x),
left(left), right(right) {}
21     * };
22     */
23 class Solution {
24 public:
25     void flatten(TreeNode* root) {
26         if(!root) return;
27         std::vector<TreeNode*> res;
28         DFS(root, res);
29         TreeNode* curr = root;
30         for(int i=1; i<res.size(); i++){
31             curr->right = res[i];//可以直接使用res[i-1]代替curr
32             curr->left = nullptr;
33             curr = curr->right;
34         }
35         curr->left = nullptr;
36         curr->right = nullptr;
37         return;
38     }
39
40 private:
41     // 前序遍历即可
42     void DFS(TreeNode* root,
43         std::vector<TreeNode*>& res){
44         if(!root) return;
45         res.push_back(root);
46         DFS(root->left, res);
47         DFS(root->right, res);
48     }
49 };
50
51
52 //-----原地实现是真的绕啊-----
53 class Solution {
54 public:
55     void flatten(TreeNode* root) {
56         backtrack(root);
57     }
58
59 private:
60     TreeNode* backtrack(TreeNode* root){
61         if(!root) return nullptr;
62         if(!root->left && !root->right){// root 为叶子节点
63             return root;// 返回 root
64         }
65         TreeNode* left = root->left;// 指向 root 的左子树
66         TreeNode* right = root->right;// 指向 root 的右子树
67         TreeNode* left_tail = nullptr;//
68         TreeNode* right_tail = nullptr;//
69         TreeNode* tail = nullptr;// 将指向root二叉树的最后一个节点
70         root->left = nullptr;// 将 root 的做指针置空
71
72         if(left){// 当 left 不为空时

```

```
73         // 递归的将左子树转为链表, left_tail 指向左子树最后一个节点
74         left_tail = backtrack(left);
75         root->right = left;
76         tail = left_tail;
77     }
78
79     if(right){
80         right_tail = backtrack(right);
81         if(left){
82             left_tail->right = right;
83         }
84         tail = right_tail;
85     }
86     return tail;
87 }
88 };
89 // @lc code=end
90
91
```